Mascheramento bitwise avanzato: AND, OR, NOT e un LED che si sposta

Lezione 5 — 4MAK, SST-INF-01c — Ripasso: perché mascherare, MSB/LSB, nuovo esercizio con shift

30/09/2026

1 Ripasso: il problema del mascheramento

Nota

Riprendiamo l'esercizio della lezione precedente: un PIC16F628A con **due pulsanti** collegati in configurazione con resistenza di **pull-up** (il pulsante è **normalmente alto**, scende a **livello basso** solo mentre viene **tenuto premuto**, poi risale quando si rilascia) e **otto LED** collegati su **PORTB**.

Attenzione

Il problema di fondo: quando si legge l'**intera PORTA**, il byte letto contiene **8 bit**, ma a noi interessano **solo 1 o 2** di quei bit (quelli collegati ai pulsanti). Gli altri bit possono contenere **valori residui** che non conosciamo e di cui **non ci fidiamo** – non possiamo scegliere cosa fare in base a un dato se non sappiamo **con certezza** cosa contiene.

Esempio

Analogia usata in aula: se qualcuno dice "ti regalo questi soldi qua" senza specificare la cifra esatta, il **regalo non è valutabile** – potrebbero essere **30 euro** o **1230 euro**, un 1 o un 2 fanno una differenza enorme. Lo stesso vale per un **numero binario**: non sapere cosa c'è nelle **posizioni che non ci interessano** ci impedisce di decidere con certezza cosa fare con l'intero numero.

Definizione

La soluzione: **moltiplicare (bit a bit)** ciascun bit del dato letto **per 0** nelle posizioni che non interessano, così da **azzerarle con certezza** – qualsiasi valore ci fosse (0 o 1), moltiplicato per 0 diventa 0. Questa tecnica si chiama **mascheramento**: usare un numero (la **maschera**) con **1 solo nelle posizioni che interessano**, per filtrare via tutti i **bit spuri**.

2 Esempio numerico: pulsanti su RA2 e RA3

Esempio

Ripetendo l'esempio con i pulsanti A e B collegati su RA2 e RA3: la **maschera** da usare è **0x0C**, cioè **00001100** in binario.
Se invece i pulsanti fossero stati collegati su **RA1 e RA3**, la maschera sarebbe stata **0x0A** (dato che 00001010 in binario corrisponde a 0x0A, con i **bit 1 e 3 a 1**) – il valore della maschera **dipende sempre da *dove*** sono **collegati fisicamente** i segnali che interessano.

Attenzione

Perché non basta confrontare PORTA con 0x0C direttamente, senza prima mascherare: se ad esempio RA7 (il bit più a sinistra) **valesse 1 invece di 0**, il valore letto sarebbe **10001100** e non più uguale a 0x0C. Un **confronto diretto** con un if fallirebbe sempre, perché **l'if confronta l'intero numero**, non solo i bit che interessano – e il numero intero **cambia a seconda di cosa c'è negli altri bit**, anche se i bit che contano per noi sono rimasti identici.

Definizione

Come funziona l'AND bitwise bit per bit: ogni bit del dato viene confrontato con il bit corrispondente della maschera usando la **tabella dell'AND logico** (1 AND 1 = 1, 1 AND 0 = 0, 0 AND 1 = 0, 0 AND 0 = 0). Con la maschera **0x0C** = 00001100:

- nelle posizioni dove la **maschera ha 0**, il risultato è **sempre 0** (qualsiasi cosa AND 0 = 0): questi bit vengono "**spenti**" con certezza;
- nelle posizioni dove la **maschera ha 1**, il risultato è **uguale al bit originale** (qualsiasi cosa AND 1 = il valore stesso): questi bit "**passano**" inalterati.

Esempio

Fatto il confronto bit a bit, il risultato dell'AND con 0x0C **può valere solo**:

- 0x0C: **nessun pulsante premuto** (bit RA2 e RA3 entrambi **alti**);
- 0x08: premuto il pulsante su **RA3**;
- 0x04: premuto il pulsante su **RA2**;
- 0x00: **entrambi i pulsanti premuti** insieme – situazione che non doveva succedere secondo la traccia, ma esiste come **quarta possibilità teorica**.

Nota

Metodo usato in aula quando un concetto non è chiaro: a questo punto della lezione, un allievo non aveva capito il ragionamento. L'insegnante non si è limitato a ripetere la stessa spiegazione, ma ha chiesto esplicitamente di **individuare *dove*** esattamente fallisse la comprensione, invitando poi l'allievo a **riformulare il concetto** con parole proprie (concedendo anche un momento per trovare il linguaggio giusto prima di rispondere): un modo di **verificare**

la comprensione reale, più efficace del semplice "hai capito?".

3 Perché la maschera evita 256 `if`

Attenzione

Senza il mascheramento, per gestire con certezza tutti i casi possibili servirebbero **256 `if` diversi** (**tutte le combinazioni di 8 bit**), di cui **solo 4** farebbero qualcosa e **252** non farebbero nulla, perché rappresentano **casi che non interessano**. Se anche una sola di queste 256 possibilità non viene prevista nel codice, il programma ha un **bug**: non è che "non si inventano le cose", semplicemente **il sistema non risponde** in quella situazione specifica.

Nota

Il mascheramento elimina questo problema: applicando la AND con la maschera, **tutte le 252 combinazioni irrilevanti collassano** sempre sugli stessi **pochi valori previsti** (`0x00`, `0x04`, `0x08`, `0x0C`), indipendentemente da cosa contengono realmente gli altri bit in quel momento. Si può quindi scrivere **codice sicuro** con **solo 4 `if`**, invece di 256.

Esempio

Generalizzazione: con **3 pulsanti** invece di 2, le combinazioni diventano 8 (perché 3 bit generano $2^3 = 8$ casistiche), quindi servirebbero **8 `if`**. Con **8 pulsanti** collegati, senza fare nessun filtro servirebbe scrivere tutte le **256 condizioni** – a meno di usare **altre tecniche più avanzate**, non trattate in questa lezione.

4 Il mascheramento al contrario: OR per forzare a 1

Definizione

Oltre alla AND (che forza a 0 i bit che non interessano), esiste un mascheramento che lavora al contrario con l'operatore **OR** (`|`): `PORTA | 0xF3` **forza a 1** tutti i bit dove la maschera ha 1, **lasciando intatti** solo i bit dove la maschera ha 0.
Con la maschera `0xF3` = `11110011`, i bit RA2 e RA3 (dove la maschera ha 0) **restano quelli che erano**, mentre tutti gli altri vengono **forzati a 1**.

Nota

Riepilogo dei due mascheramenti:

- **AND con una maschera** (1 nelle posizioni che interessano): forza a 0 tutto il resto.
- **OR con una maschera** (0 nelle posizioni che interessano): forza a 1 tutto il resto.

5 L'operatore NOT (tilde) per scrivere la maschera OR più leggibile

Definizione

I programmatori di firmware **raramente** scrivono `0xF3` a mano: per ottenerlo si dovrebbe fare il conto del **complemento** a partire da `0x0C`, un'operazione che richiede di avere in mente tutta la tabella della conversione. È molto più leggibile scrivere invece **`~0x0C`** (tilde davanti al numero): l'operatore **NOT bitwise** (~) **inverte ogni singolo bit** del numero (**ogni 0 diventa 1** e **ogni 1 diventa 0**).
Partendo da `0x0C` = `00001100` e applicando il NOT si ottiene **`11110011`**, cioè esattamente **`0xF3`** – ma scritto in un modo che si legge e si ragiona più facilmente, perché si parte da un **numero già familiare** (la maschera AND già vista) invece di doverne calcolare uno nuovo a mente.

Esempio

Un caso d'uso della combinazione AND + NOT: scrivere `PORTA AND (~0x0C)` equivale a **forzare a 0** proprio i bit **RA2 e RA3** (quelli che normalmente ci interessano), lasciando intatti tutti gli altri – è l'**operazione opposta** rispetto al mascheramento normale. Questo può essere utile per "**disabilitare**" artificialmente la lettura di certi pulsanti in una fase in cui la macchina non deve rispondere a quei comandi, anche se sono **casi d'uso piuttosto particolari** e non la normalità.

6 Cifra più significativa: l'analogia del regalo in denaro

Esempio

Per introdurre il concetto di **bit più significativo**, è stata ripresa l'analogia del **regalo in denaro**: se si regala una cifra scritta con delle posizioni (es. **unità**, decine, centinaia, migliaia), la cifra nella **posizione più a sinistra** "**vale di più**" delle altre – perdere lo zero nella posizione delle migliaia (passando da 9.000 a 900, ad esempio) fa perdere una somma molto più grande che perdere una cifra nella posizione delle unità.

Definizione

Principio generale: la posizione che una cifra occupa in un numero ne determina il **peso** – **più una cifra sta a sinistra, più vale**. Confrontando due numeri scritti con le stesse cifre ma in ordine diverso, è sempre quello con la **cifra più "pesante"** nella posizione più a sinistra ad essere maggiore.
Lo stesso principio vale per i bit di un byte: ogni posizione ha un **peso** (0, 1, 2, 3, ... per le posizioni da destra verso sinistra), e i **bit più a sinistra** "**pesano**" di più.

7 MSB e LSB

Definizione

Applicando questo principio ai bit: il **bit più a sinistra** di un byte si chiama **MSB** (*Most Significant Bit*, il bit più significativo), mentre il **bit più a destra** si chiama **LSB** (*Least Significant Bit*, il **bit meno significativo**, quello che vale di meno).

Nota

Queste due sigle (MSB e LSB) compaiono molto spesso nei **datasheet** dei microcontrollori, quindi è importante **riconoscerle**.

Esempio

Un caso pratico dove questa distinzione conta: alcuni microcontrollori non lavorano internamente a 8 bit ma, ad esempio, a **6 bit** – tuttavia il dato finale viene comunque inserito in un **contenitore a 8 bit**. In questo caso si pone il problema di come posizionare i bit **"mancanti"**: **giustificare a sinistra** (i bit significativi occupano le posizioni più a sinistra, quelle non usate restano a destra) oppure **giustificare a destra** (il contrario). Entrambe le scelte sono **valide a seconda del contesto**, l'importante è ragionare su quale abbia senso per l'uso specifico che se ne deve fare.

Esempio

Applicazione pratica, riprendendo l'esercizio dei pulsanti A/B su RA2/RA3 della lezione precedente: una volta ottenuto `buttons` con la **maschera `0x0C`**, si può scrivere la **logica completa** alla **lavagna**:

```
while (1)
{
    char buttons = PORTA & 0x0C;

    if (buttons == 0x0C)      // nessun pulsante premuto
    {
        PORTB = 0x00;
    }
    if (buttons == 0x08)      // pulsante B: LED pari
    {
        PORTB = 0x55;
    }
    if (buttons == 0x04)      // pulsante A: 4 LED piu' significativi
    {
        PORTB = 0xF0;
    }
}
```

`0x55` (`01010101` in binario) accende i LED nelle **posizioni pari** (0, 2, 4, 6); `0xF0` (`11110000` in binario) accende i **4 bit più significativi** (RB7-RB4), lasciando spenti i **4 meno significativi** – lo stesso principio del **MSB** appena visto, applicato concretamente ai LED.

8 Nuovo esercizio guidato: un LED che si sposta con due pulsanti

Esempio

Traccia (testo esatto scritto in cima al programma): "Scrivere un programma che **rilevi due pulsanti** collegati rispettivamente su **RA1 e RA4**. Il circuito usa **8 led** collegati su **PORTB**. Solo **1 led rimane sempre acceso**. All'accensione il Led acceso è **RB4**. Se premo RA1, il led **scende** (RB3 → RB2 → RB1 → RB0). Se premo RA4, il led **sale** (RB4 → RB5 → RB6 → RB7). Ogni spostamento è sempre di **un solo bit**."

Nota

Analogia usata in aula: il comportamento è simile a quello di un **termostato digitale**, dove un **LED indica la temperatura impostata** – premendo "più" la temperatura (e quindi la posizione del LED) sale, premendo "meno" scende, ma sempre di **un passo alla volta**, **mai con salti**.

Attenzione

Discrepanza osservata tra la specifica e il codice realmente scritto: la specifica sopra dichiara "RA1 scende, RA4 sale", ma il codice finale realizzato in aula (visibile a schermo durante la lezione, poi copiato identico anche su altre postazioni) ha i **due pulsanti invertiti** rispetto a questa dichiarazione: è il pulsante su **RA1** (maschera `0x02`) a far ***salire*** il LED con lo **shift a sinistra**, ed è il pulsante su **RA4** (maschera `0x10`) a farlo ***scendere*** con lo **shift a destra**. Questa dispensa riporta fedelmente il codice realmente scritto e funzionante (comprensivo dei **commenti originali** `// Sale` e `// Scende` accanto ai due `if`), non la specifica iniziale: in caso di dubbio in laboratorio, **verificare sul circuito reale** quale pulsante fa cosa, dato che le due fonti sono in **contraddizione** tra loro.

8.1 Perché serve il delay

Attenzione

Il modo più semplice per realizzare questo comportamento usa i **delay** (**ritardi**): quando si preme fisicamente un pulsante, anche se la pressione dura solo una frazione di secondo, il microcontrollore è **talmente veloce** che "vede" il pulsante premuto **centinaia di volte** in quell'intervallo. Senza un ritardo dopo ogni cambiamento, il LED si sposterebbe **istantaneamente** fino al limite massimo o minimo a ogni singola pressione, invece di avanzare di **un solo passo**. Inserendo un ritardo dopo ogni spostamento, si dà il tempo alla persona di **rilasciare il dito** prima che il programma legga di nuovo lo stato del pulsante – questo rallenta leggermente il funzionamento della macchina, ma è un **compromesso accettabile**.

8.2 Impostazione del progetto

Definizione

Come nell'esercizio precedente, il primo passo è impostare i **Configuration Bits** da MPLAB X (menu *Window* → *Target Memory View* → *Configuration Bits*): **oscillatore** di tipo **HS** (**quarzo esterno**), **watchdog spento**, **power-up timer attivato**, **LVP spento**. Si copia il codice **`#pragma config`** generato automaticamente in cima al file.
Essendo certi di dover usare i delay, si definisce subito anche la **frequenza del quarzo** con `#define` **`_XTAL_FREQ`**, impostata a **20 milioni** (20 MHz).

8.3 Configurazione di TRISA e TRISB

Esempio

I pulsanti sono collegati su **RA1** e **RA4**, quindi questi due bit di `TRISA` vanno **impostati a 1** (input), tutti gli altri **a 0**. Scrivendo il numero in binario come **`0b76543210`** (posizioni da 7 a 0), i bit 4 e 1 a 1 danno **`00010010`**, che in esadecimale si scrive **0x12** (perché `0001` corrisponde a 1 e `0010` corrisponde a 2, nelle rispettive metà del byte).
`TRISB` va invece impostato interamente a **output**, quindi **0x00**, perché tutti gli 8 piedini di PORTB pilotano i **LED**.

8.4 La variabile di stato per il LED

Attenzione

Scrivere direttamente `PORTB = 0x10` per accendere il LED centrale all'avvio funzionerebbe, ma è una **pratica rischiosa**: il dato sarebbe memorizzato solo nel **registro fisico** `PORTB`, e se per qualche motivo un bit di PORTB venisse successivamente **configurato come input** invece che output, si perderebbe il riferimento a quale LED dovrebbe essere acceso, con il rischio di **comportamenti imprevedibili**.

Definizione

Soluzione più robusta: creare una **variabile** **`char`** (chiamata ad esempio **`led`**) che rappresenta la **configurazione desiderata** dei LED, inizializzata a **`0x10`** (RB4 acceso). All'avvio si scrive **`PORTB = led`**. Da questo punto in poi, tutto il ragionamento sullo spostamento del LED lavora sulla variabile `led`, e solo alla fine il suo valore viene **copiato su** `PORTB` – **lavorare su una variabile propria**, anziché direttamente sul registro hardware, **protegge il programma** da eventuali interferenze esterne o errori di configurazione successivi che potrebbero alterare il valore fisico della porta.

8.5 Lettura e mascheramento dei pulsanti

Esempio

Dentro il **ciclo** `while(1)`, si legge lo stato dei pulsanti **mascherando** `PORTA` con **`0x12`** (lo stesso valore usato per configurare **`TRISA`**, perché sono proprio i bit di RA1 e RA4 che interessano):

```
char buttons = PORTA & 0x12;
```

Le combinazioni possibili di `buttons` sono **quattro**: **nessun pulsante premuto**, **solo quello su RA1**, **solo quello su RA4**, o **entrambi insieme** (caso che la traccia non prevede esplicitamente, ma che in pratica dà **0x02** se è premuto solo RA1 e **0x10** se è premuto solo RA4).

8.6 Shift a sinistra e a destra per muovere il LED

Definizione

Per spostare il LED acceso di una posizione si usa l'operatore di **shift** del linguaggio C (già visto in precedenza), che permette di spostare tutti i bit di un byte **verso sinistra o verso destra** di un **numero scelto di posizioni**.

Esempio

Se è premuto il pulsante che fa "**salire**" il LED (RA1, valore **`0x02`**, commentato **`// Sale`** nel codice originale), si scrive:

```
if (buttons == 0x02) // Sale
{
    led = led << 1;
    PORTB = led;
    __delay_ms(500);
}
```

Se invece è premuto il pulsante che fa "**scendere**" il LED (RA4, valore **`0x10`**, commentato **`// Scende`** nel codice originale), il codice è identico ma con lo shift nella **direzione opposta**:

```
if (buttons == 0x10) // Scende
{
    led = led >> 1;
    PORTB = led;
    __delay_ms(500);
}
```

Il valore del **delay** (**500 ms**, scelto largo per sicurezza) può in seguito essere **ridotto**, purché il funzionamento resti corretto – un tempo **troppo breve** rischierebbe di non dare abbastanza **margine** all'utente per rilasciare il pulsante.

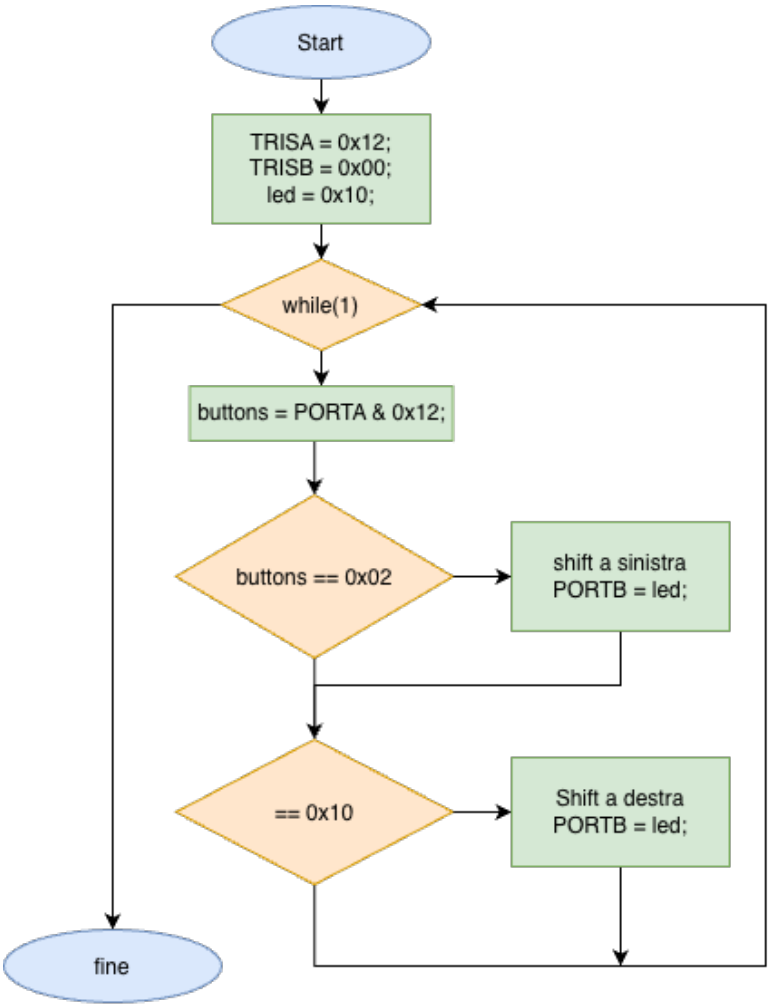


Figura 1: Diagramma di flusso della prima versione (senza controllo dei limiti): il rombo `while(1)` rappresenta il test del ciclo stesso, sempre vero nella pratica (il ramo "No" verso **fine** non viene mai percorso, dato che il firmware non deve mai terminare). Dentro al ciclo, i due `if` sono indipendenti nel codice C: anche quando il primo è vero (shift a sinistra eseguito), il flusso passa comunque dal secondo controllo prima di rientrare nel test `while(1)`, fedele all'ordine di esecuzione reale del codice. Manca ancora il controllo dei bordi del range, introdotto nella versione corretta più avanti.

Nota

Warning del compilatore osservato durante la prova: compilando questo codice compare un **warning** (**non un errore**) che segnala una **conversione implicita di tipo**: scrivendo `led << 1`, il numero `1` viene trattato dal compilatore come **`int`**, mentre `led` è dichiarato come **`char`**, quindi il compilatore segnala la conversione implicita tra i due tipi. In questo caso specifico il warning non causa alcun problema pratico e **può essere ignorato**, ma è importante saperlo riconoscere per **non confonderlo con un errore reale**.

8.7 Il problema dei limiti: cosa succede ai bordi

Attenzione

Provando il programma **sul circuito reale**, lo spostamento funziona correttamente nella parte centrale del range, ma emerge un problema ai limiti: se si continua a premere il pulsante che fa salire il LED **oltre la posizione RB7** (l'ultima disponibile, valore **`0x80`** in binario `10000000`), il **bit più a sinistra viene perso** nello shift, il **LED si spegne del tutto**, e anche tornando indietro con l'altro pulsante il LED **non si riaccende più** – il programma, a quel punto, ha "**dimenticato**" dove si trovava.

Definizione

Causa: lo shift a sinistra, applicato a un valore che ha già il bit più significativo (**MSB**) acceso, fa **"uscire" quel bit dal byte**, perdendolo per sempre: il risultato diventa **`0x00`**, cioè **tutti i LED spenti**, una situazione da cui il codice (così com'è) **non può più recuperare**.

Esempio

Correzione discussa in aula: prima di eseguire lo shift a sinistra, bisogna **controllare** che il valore di `led` sia ancora **inferiore al limite massimo** (**`0x80`**); analogamente, prima di uno shift a destra, bisogna controllare che `led` sia **superiore al limite minimo** (**`0x01`**). In altre parole: quando si sale, si controlla un limite **"minore di"**; quando si scende, si controlla un limite **"maggiore di"** (la logica **si inverte** a seconda della direzione del movimento):

```
if (buttons == 0x02 && led < 0x80) // Sale
{
    led = led << 1;
    PORTB = led;
    __delay_ms(500);
}
if (buttons == 0x10 && led > 0x01) // Scende
{
    led = led >> 1;
    PORTB = led;
    __delay_ms(500);
}
```

Con questo controllo aggiuntivo, il LED **non esce più dal range valido**: arrivato al **bordo** (RB7 o RB0), ulteriori pressioni dello stesso pulsante semplicemente **non hanno più effetto**, finché non si preme il **pulsante opposto**.

9 Da ricordare

- Il **mascheramento bitwise** (operatore AND con una maschera che ha 1 solo nelle posizioni che interessano) **isola i bit rilevanti** da un byte, **azzerando con certezza** tutti gli altri: evita di dover scrivere **centinaia di `if`** per gestire ogni possibile combinazione dei bit che non interessano.
- Il mascheramento **OR** (con una maschera che ha 1 nelle posizioni da forzare) fa l'operazione

opposta: **forza a 1** i bit scelti.

- L'operatore **NOT bitwise** (~) **inverte tutti i bit** di un numero, ed è usato spesso per scrivere in modo più leggibile una **maschera OR** a partire dalla **maschera AND** corrispondente (es. `~0x0C` invece di calcolare a mano `0xF3`).
- **MSB** (*Most Significant Bit*) è il **bit più a sinistra** di un byte (quello che "pesa" di più), **LSB** (*Least Significant Bit*) è quello **più a destra**: sigle molto comuni nei **datasheet** dei microcontrollori.
- Gli operatori di **shift** (<< per sinistra, >> per destra) **spostano tutti i bit** di un valore, e sono utili per implementare uno **stato che si muove** (come un LED che scorre) – ma vanno sempre protetti con un **controllo dei limiti** (< o > rispetto al valore massimo/minimo), altrimenti un bit può "uscire" dal byte e **andare perso**.
- I **delay** sono necessari ogni volta che si legge un **pulsante fisico** dentro un ciclo, per evitare che una singola pressione, durata solo una **frazione di secondo**, venga interpretata come **centinaia di pressioni consecutive** dal microcontrollore.